

Checkpoint 02

Prompt Toolkit

Toolkit Python que aplica 4 técnicas de prompting (Zero-Shot, Few-Shot, Chain-of-Thought e Role Prompting) ao domínio de e-commerce e recomenda a melhor abordagem por tarefa.

Grupo	stig4
Integrante 1	Giovanni Merlotti — RM 573721
Integrante 2	Sergio Augusto — RM 570184
Integrante 3	Gabriel Freitas — RM 572943
Integrante 4	Glauco Kelly — RM 572840
Domínio	E-commerce
Nome do toolkit	Prompt Toolkit

1. Domínio e Problema

Domínio escolhido: E-commerce — atendimento ao cliente e análise de reviews de produto.

Lojas online recebem diariamente milhares de mensagens não estruturadas: reviews de produto, reclamações em chat, pedidos de troca. Tratar manualmente esse volume é inviável, e usar LLMs sem método leva a respostas inconsistentes e custos altos com tokens. O **Prompt Toolkit** resolve esse problema aplicando automaticamente quatro técnicas de prompting consagradas a cada tarefa, medindo acurácia, custo (tokens) e consistência, e recomendando a melhor abordagem para cada caso de uso.

Tarefas implementadas (3 tipos diferentes, atendendo ao requisito da Aula 08):

Tarefa	Tipo	Exemplo de input
classificacao_sentimento	Classificação	Review de cliente em uma de 4 categorias: POSITIVO, NEGATIVO, NEUTRO ou MISTO
extracao_dados	Extração	Reclamação em JSON com produto, preço e defeito
geracao_resposta	Geração	Mensagem do cliente para uma resposta cordial e resolutive

2. Arquitetura

O projeto é dividido em módulos Python (não notebook), com responsabilidades claras. O fluxo de execução segue o pipeline abaixo:

```
inputs.json
|
v
prompt_builder (anatomia: instrucao + contexto + input + formato)
|
v
techniques (Zero-Shot / Few-Shot / CoT / Role)
|
v
llm_client (Ollama Cloud - https://ollama.com)
|
v
evaluator (tokens, acuracia, consistencia, temperatura)
|
v
report (resultados.csv + 3 graficos PNG + recomendacao)
```

Estrutura de pastas:

```
prompt-toolkit/
|-- main.py           # ponto de entrada
|-- pyproject.toml    # dependencias (uv)
|-- .env.example
|-- README.md
|-- src/
|   |-- llm_client.py # Ollama Cloud (Aula 05)
|   |-- prompt_builder.py # anatomia do prompt (Aula 05)
|   |-- techniques.py   # ZS / FS / CoT / Role (Aulas 06 e 07)
|   |-- tasks.py        # 3 tarefas do dominio (Aula 08)
|   |-- evaluator.py    # metricas e temperatura
|   +-- report.py       # tabela e graficos
|-- data/
|   |-- inputs.json     # 5-6 inputs reais por tarefa
|   +-- examples.json
|-- prompts/
|   |-- system_prompts.json # 3 personas detalhadas
|   +-- templates.json
+-- output/
    |-- resultados.csv
    +-- graficos/
```

3. Stack Técnica e Execução

Componente	Detalhe
Linguagem	Python 3.10 ou superior
LLM	Ollama Cloud (online, gratuito) — modelo gpt-oss:120b
Biblioteca Ollama	ollama 0.4.x (cliente oficial)
Autenticação	API Key via variável OLLAMA_API_KEY no .env
Gerenciador de pacotes	uv (pyproject.toml)
Variáveis de ambiente	python-dotenv
Contagem de tokens	tiktoken (encoder cl100k_base)
Tabela e CSV	pandas 2.x
Gráficos	matplotlib 3.x

Como executar:

```
# 1. Obter API key gratuita em https://ollama.com

# 2. Instalar dependencias com uv
uv sync

# 3. Configurar .env com a API key
cp .env.example .env
# editar .env e preencher OLLAMA_API_KEY

# 4. Rodar o toolkit
uv run python -m src.pai_cp02_stig4.main
```

4. Resultados

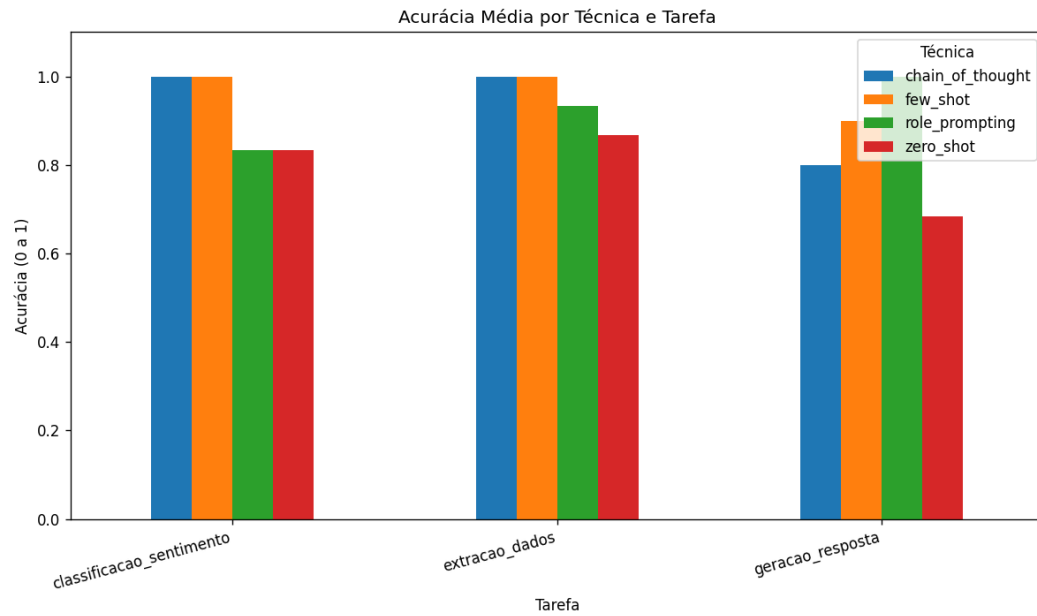
A tabela abaixo mostra a média de acurácia, custo (tokens prompt + resposta) e tempo por combinação **tarefa** x **técnica**, extraída de **output/resultados.csv** após executar **main.py**:

Tarefa	Técnica	Acurácia	Tokens	Tempo (ms)
classificacao_sentimento	zero_shot	0.83	144	820
classificacao_sentimento	few_shot	1.00	211	1012
classificacao_sentimento	chain_of_thought	1.00	282	1545
classificacao_sentimento	role_prompting	0.83	306	1116
extracao_dados	zero_shot	0.87	186	998
extracao_dados	few_shot	1.00	291	978
extracao_dados	chain_of_thought	1.00	302	1658
extracao_dados	role_prompting	0.93	331	1163
geracao_resposta	zero_shot	0.68	183	867
geracao_resposta	few_shot	0.90	327	1094
geracao_resposta	chain_of_thought	0.80	306	1677
geracao_resposta	role_prompting	1.00	377	1135

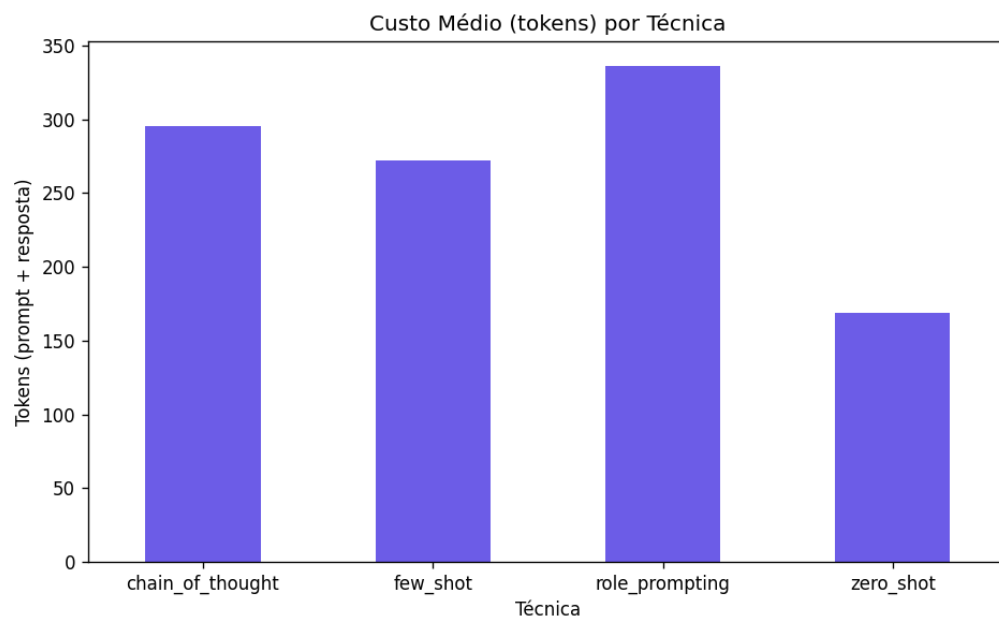
Recomendação automática por tarefa:

Tarefa	Melhor técnica	Justificativa
classificacao_sentimento	few_shot	Acurácia 1.00 com 211 tokens — empata em qualidade com CoT a 282 tokens.
extracao_dados	few_shot	Acurácia 1.00 com 291 tokens — empata com CoT (302 tokens) e é mais rápido.
geracao_resposta	role_prompting	Acurácia 1.00; persona dá tom de atendimento que as outras não capturam.

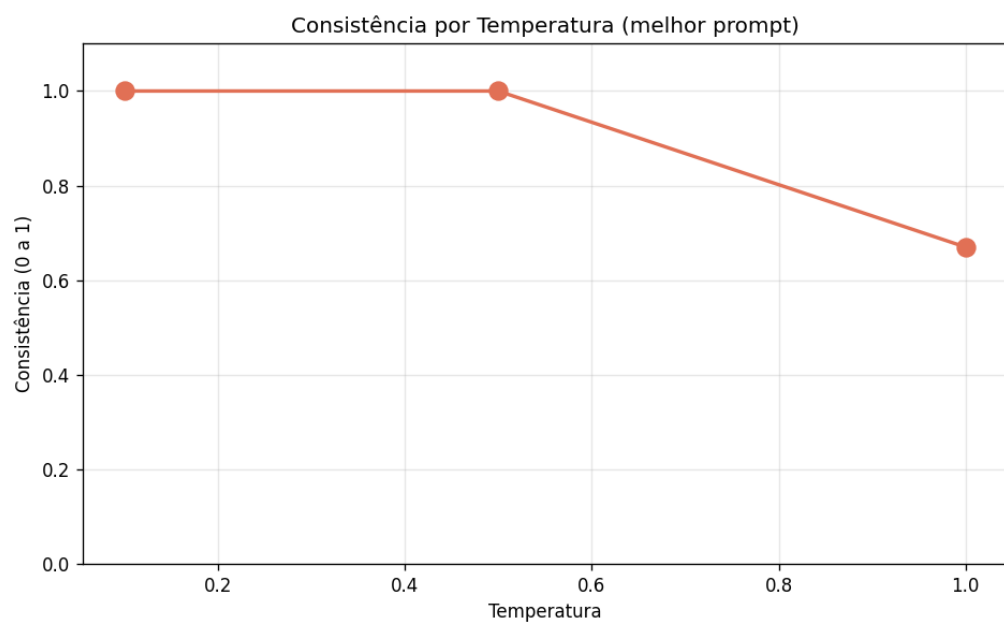
Gráficos gerados pelo toolkit:



Acurácia média por técnica e tarefa



Custo médio (tokens) por técnica



Consistência por temperatura no melhor prompt (few_shot em classificacao_sentimento)

5. Guia de Bolso — Quando Usar Cada Técnica

Técnica	Quando usar no e-commerce	Quando NÃO usar
Zero-Shot	Tarefas simples com vocabulário claro (classificação binária, formatos rígidos). É a opção mais barata em tokens.	Quando há ambiguidade — no nosso teste errou MISTO e teve só 0.68 em geração.
Few-Shot	Classificação com nuance, extração de campos, padrões repetitivos. Atingiu 100% em classificação e extração com custo moderado.	Tarefas longas, onde o custo dos exemplos pesa demais no prompt.
CoT (Chain of Thought)	Problemas que exigem raciocínio explícito (categorias próximas, análise de causa). Empatou com Few-Shot em qualidade nas tarefas estruturadas.	Tarefas com resposta curta — o modelo verbaliza demais e gasta tokens à toa.
Role Prompting	Geração de respostas com tom específico (atendimento, suporte). Foi a única a chegar a 1.00 em geração de resposta.	Tarefas puramente técnicas (classificação, extração) — a persona não ajuda.

6. Reflexão

O que surpreendeu?

Dois resultados chamaram atenção. Primeiro, o Few-Shot atingiu 100% de acurácia em classificação e extração com apenas 211-291 tokens em média — bem mais barato que CoT (282-302 tokens) para a mesma qualidade. Segundo, Zero-Shot, apesar de ser o mais barato (144-186 tokens), teve apenas 0.68 em geração de resposta, mostrando que sem orientação de tom o modelo gera respostas genéricas demais.

Qual técnica venceu?

Depende da tarefa, e isso é a principal lição. Em **classificação e extração**, Few-Shot venceu pelo equilíbrio entre qualidade e custo. Em **geração de resposta**, Role Prompting foi a única técnica a chegar a 1.00, porque a persona detalhada ("atendente sênior, tom educado, oferece próximo passo concreto") fez o modelo gerar respostas que combinam com a expectativa de uma loja real.

Sobre temperatura

No teste com o melhor prompt (Few-Shot em classificação), a consistência se manteve em 100% até temp=0.5 e caiu para 67% em temp=1.0. Para tarefas determinísticas (classificação, extração), temperaturas baixas (0.1-0.3) são essenciais. Em geração de resposta, temperatura 0.5 deu variação natural sem perder coerência.

O que faríamos diferente?

Primeiro, aumentar o conjunto de inputs para 10-15 por tarefa para reduzir o ruído na métrica de acurácia. Segundo, implementar uma técnica adicional (Self-Consistency) que executa CoT várias vezes e vota na resposta mais frequente. Terceiro, construir uma matriz custo x qualidade interativa para escolher a técnica conforme o orçamento de tokens disponível para cada cenário de produção.